

卡比冒险之旅

基于 Qt6 的 2D 横板动作冒险游戏 课程作业报告

组号：2

小组成员：彭延睿、张良臣、王思钺

开发环境：Qt 6.11 + MinGW 64-bit + C++17

开发工具：Qt Creator

一、程序功能介绍

本作品是一款基于 C++ 与 Qt6 框架开发的 2D 横板动作冒险游戏，灵感来源于任天堂《星之卡比》系列。玩家操控卡比（Kirby）在横版地图中冒险闯关，通过跳跃、攻击、吞噬敌人获得复制能力，挑战各具特色的 Boss，最终抵达终点。

1.1 核心玩法

游戏提供两大模式：(1) 冒险模式——在长达 52000 像素的横版地图中探索前进，越过草地、泥土、水体、冰面、尖刺等复杂地形，躲避并击败各类敌人，收集能力，最终到达终点旗帜；(2) Boss 挑战模式——玩家可选择挑战三位 Boss，每位 Boss 均拥有独立的多阶段战斗机制和弹幕攻击模式。

1.2 玩家能力系统

卡比拥有丰富的动作系统：行走、跳跃（含二段跳）、漂浮、翻滚闪避、近战攻击、吞噬敌人、吐出子弹等。击败带有特殊属性的敌人后可通过吞噬消化获得其复制能力，包括：

- (1) 火形态（FIRE）——疾跑冲刺、自爆攻击，对冰面有融化效果
- (2) 冰形态（ICE）——冻结水面行走、防御护盾，克制火系敌人
- (3) 叶形态（LEAF）——远程回旋镖攻击，持续范围伤害
- (4) 电形态（SPARK）——高速飞行、闪电冲刺，可探索隐藏区域

不同形态之间存在元素相克关系。能力池最多可存储两种能力，通过 R 键随时切换。每种形态均有独立的待机/行走/跳跃/翻滚/攻击动画序列。若长时间不切换，形态将在 30 秒后自动消失。

1.3 关卡与环境交互

关卡采用基于字符矩阵的 Tile 地图系统，支持草地、泥土、水体（游泳消耗体力）、冰面（打滑效果）、碎石、多种尖刺（固定刺、移动刺、伏击刺）等方块类型。水体与冰面可通过冰形态能力冻结/融化，木箱可推动，蛋糕可拾取获得临时攻击力加成和星星子弹储备。游戏实现了检查点存档和终点通关机制。冒险模式地图自动优化，包括填平坑洞和缩短过长平地。

1.4 Boss 战系统

游戏包含三位独特设计的 Boss：

- (1) 克苏鲁之脑（Brain Of Cthulhu）——飞行弹幕型 Boss，满血 400 点。一阶段飞行追踪并发射弹幕，半血后进入二阶段传送突袭模式，大幅提升攻击频率。
- (2) 猪鲨公爵（Duke Fishron）——三阶段变身型 Boss，满血 400 点。一阶段蓄力冲刺，二阶段变身并切换攻击模式（螺旋弹幕+直线扇形弹幕+地形水转换），三阶段三连冲刺速度大幅提升。
- (3) 冰雪之神（IceGod）——召唤型 Boss，满血 400 点。可召唤旋转雪花护卫（Xuehwa）并释放三种攻击：冰锥扩散、瞄准冰束、冰锥雨。

每位 Boss 均配有独立的登场动画、战斗音乐、血条 HUD 和死亡白屏徽章掉落机制。

1.5 音画与 UI 系统

画面方面，游戏采用了四层视差滚动背景（Clouds 1-4 四套云层 PNG 序列），Boss 二阶段可切换至 Clouds 3 背景并做透明渐变过渡。所有角色和方块素材均采用精灵表（Sprite Sheet）切割加载。角色动画支持形态专属帧序列。

UI 系统包含：主菜单（新游戏/继续/选关/设置）、选关界面（冒险模式/Boss 挑战子菜单）、暂停菜单、设置界面（音量滑动调节）、游戏结束界面、结束动画。HUD 系统显示玩家生命值（心形图标）、体力条（水下计时）、Boss 血条（外框+红色填充）、技能冷却图标。

音效方面，BGM 使用 QMediaPlayer 循环播放，冒险/选关/Boss 战场景自动切换音乐。脚步和水声使用独立循环音道，攻击/受伤/POP 声通过短音效播放器触发，Boss 登场配有专门的 horror 音效。

二、项目各模块与类设计细节

2.1 整体架构

项目采用 Qt6 的 QGraphicsScene/View 图形框架，以 60FPS（16ms 间隔）定时器驱动游戏主循环。游戏引擎采用面向对象设计，通过 GameObject 基类派生所有游戏实体，MainWindow 作为核心控制器管理所有子系统的协调。场景尺寸设计为 52000x1200 像素（冒险模式），窗口固定为 1000x700 像素。

2.2 核心类层次结构

(1) GameObject（抽象基类）

继承自 QGraphicsPixmapItem，提供像素图显示基础能力。成员包括 vx（水平速度）、vy（垂直速度），定义纯虚函数 updateLogic()。所有游戏对象通过此方法实现每帧逻辑更新，构成多态更新架构。

(2) Player（玩家类）

继承自 GameObject，是游戏中最复杂的类，包含以下子系统：

- 动作状态机：IDLE, WALKING, JUMPING, ROLLING, ATTACKING, SWALLOWING, FATTY_IDLE, FATTY_WALKING, SPITTING, DIGESTING 共 10 种状态
- 元素属性系统：NONE/FIRE/ICE/LEAF/SPARK 四种复制能力及无能力基础状态
- 技能系统：fireSprint（火形态疾跑冲刺）、explosion（火形态自爆）、iceDefend（冰形态防御护盾）、leafSkill（叶形态回旋镖）、lightningDash（电形态闪电冲刺）
- 属性管理：hp（生命值，初始 5 点）、stamina（水下体力，300 帧约 5 秒）、cakeAmmo（蛋糕子弹）
- 状态效果：invulnTimer（受伤无敌帧）、attackPowerTimer（攻击力增强时限）、damageFlashTimer（受伤红闪）、slowTimer（减速效果）、formTimer（形态存续计时）
- 动画系统：每个形态均包含独立的 idle/walk/jump/roll/attack 帧序列，通过帧计数器驱动切换

(3) Enemy 类体系

Enemy 基类包含 hp、damage、isDead、ability（复制能力标签）等通用属性，定义 CopyAbility 枚举（NONE/FIRE/WATER/ICE/SPARK/LEAF），以及虚函数 takeDamage() 和 canBeSwallowed()。

BossEnemy 派生自 Enemy，设置 ignoresTiles=true（无视地形碰撞），canBeSwallowed()=false（不可被吞噬），作为所有 Boss 的基类。

Boss 具体实现：

- DukeFishron（猪鲨公爵）：三阶段 AI 行为树，状态机含 FLYING/PREPARING/CHARGING/COOLDOWN/PHASE_TRANSITION/PHASE_TWO/PHASE_THREE/DYING。一阶段飞行追踪+蓄力冲刺；二阶段变身切换螺旋弹幕和直线扇形弹幕，可将地形转换为水体；三阶段三连高速冲刺（maxCharges=3），速度和加速度大幅提升。
- BrainOfCthulhu（克苏鲁之脑）：二阶段 AI，半血后传送突袭。FLYING 状态下追踪玩家位置，ATTACKING 状态以 0.3 秒间隔连续发射弹幕。半血触发 PHASE2_TELEPORT，随机传送至玩家附近并高速追击。
- IceGod（冰雪之神）：多技能 Boss，可召唤专属单位。FLYING 飞行追踪；SUMMONING 召唤 Xuehua 护卫绕轨道旋转发射冰锥；ICE_SPREAD 圆形范围冰锥；ICE_BEAM 瞄准冰束；ICE_RAIN 冰锥雨覆盖攻击。

MinionEnemy（小怪类）：对应火/冰/叶/电四种属性巡逻小兵，实现巡逻 AI（来回行走+边界/悬崖检测），具备属性专属远程攻击能力（火焰弹、冰锥、回旋镖、闪电冲刺）。

Apple（苹果怪）：周期性膨胀缩小的陷阱型敌人，实现 4 阶段状态机（IDLE > WARNING > GROWING > SHRINKING），膨胀至 2.5 倍后爆发。

Xuehua（雪花护卫）：IceGod 召唤的单位，以轨道环绕 IceGod 飞行，每 60 帧（1 秒）发射一枚冰锥。

(4) Tile（方块类）

枚举 8 种类型：Grass, Dirt, WaterSurface, WaterBody, IceBlock, RubbleBlock, Spike, AmbushSpike。重写 shape() 方法，尖刺返回三角形碰撞体积实现精确判定。运动系统支持水平/垂直双轴移动（用于移动刺）。伏击刺实现完整状态机：检测玩家接近 > 预警抖动 > 高速冲刺 > 停顿 > 回缩复位。changeType() 支持动态类型转换（如冰面冻结/融化交互）。

(5) Projectile（弹幕类）

支持自定义贴图、任意方向速度向量。关键属性包括 `damage`、`lifeTime`、`hurtsPlayer/hurtsEnemies`、`hasGravity`、`ignoresWalls` 等。内置动画帧系统和帧切换计时器。`isStar` 标签支持星星子弹回收机制，`causesSlow` 支持减速效果弹幕。

(6) 道具与交互类

- Cake（蛋糕）：拾取后获得攻击力加成和星星子弹储备（5 发）
- Checkpoint（检查点）：激活后更新存档复活位置，使用双态贴图区分激活状态
- Goal（终点旗帜）：触碰后触发通关判定
- Crate（木箱）：可推动的物理交互对象，配 0.5 秒平滑缓动动画（`CratePushAnim`）
- Star（星星）：Boss 战中周期性生成，`BossStarSpawnTimer` 每 180 帧生成一颗
- Badge（徽章）：Boss 死亡后掉落，含重力下落物理，分猪鲨/克苏鲁/冰雪女王三种类型

2.3 MainWindow（核心控制器）

`MainWindow` 继承自 `QMainWindow`，是游戏的中央控制器，职责涵盖：

- 游戏状态机：管理 9 种状态（`START_SCREEN`、`LevelSelect`、`INTRO_PAN`、`PLAYING`、`PAUSED`、`SETTINGS`、`MAIN_MENU`、`GAME_OVER`、`ENDING`），通过 `switch` 语句分发每帧逻辑
- 关卡加载系统：`loadLevel(int levelNum)` 方法解析字符矩阵生成 `Tile` 地图，支持冒险模式（52000 像素）和 Boss 战模式
- 主游戏循环：`gameUpdate()` 槽函数每 16ms 执行一次，驱动所有游戏对象的 `updateLogic`、碰撞检测、相机更新、HUD 更新
- 碰撞检测系统：玩家与 `Tile` 方块的四方向碰撞、玩家与敌人的伤害判定、弹幕碰撞检测、道具拾取判定
- 相机系统：平滑跟随玩家滚动，以 `actualCameraX/Y` 实现边界钳制，`screenShakeTimer` 实现屏幕震动
- UI 管理：完整的 UI 栈，提供 `showMainMenu/showLevelSelect/showSettings/showGameOver/startEndingAnimation` 等方法
- 音效管理：BGM 切换逻辑、多通道音效协调、音量控制（`QAudioOutput::setVolume`）
- Boss 战管理：Boss 登场动画（红色遮罩+白色感叹号+horror 音效）、血条 HUD、死亡白屏特效、徽章掉落机制
- 存档管理：保存/读取关卡进度、能力列表、HP、体力等状态

2.4 关卡地图系统

关卡由字符矩阵定义，每一字符对应一种方块类型：1=草地、2=泥土、3=水面、4=水体、5=冰面、6=碎石。特殊字符：P=玩家出生点、C=检查点、T=终点旗帜、I=冰小怪、F=火小怪、G=叶小怪、L=电小怪、E=苹果怪、K=蛋糕、B=木箱。冒险模式地图自动进行地形优化（填平单个坑洞、缩短过长平地并交替插入碎石和空气砖）。Boss 战地图为 80 列平地地图，两侧设置透明空气墙防止越界。

2.5 物理与碰撞系统

实现了基本的 2D 物理引擎：重力加速度（0.6/帧）、水平/垂直速度、与 `Tile` 的四方向碰撞检测（下向落地、上向撞头、侧向推挤）。玩家在水中受浮力影响，跳跃高度降低，同时体力值持续消耗，归零后扣血。冰面具有低摩擦系数，行走打滑。各形态能力改变物理参数（如火形态疾跑速度翻倍、电形态可飞行）。翻滚提供短暂无敌帧，可用于躲避攻击和穿越敌人。

2.6 动画与音频系统

画面方面，游戏采用四层视差滚动背景（`Clouds 1-4 PNG` 序列），根据相机位置以不同速率平移产生景深感。Boss 二阶段切换至 `Clouds 3` 背景并做 60 帧透明渐变过渡。角色动画通过精灵表切割加载，每个形态 5-8 帧，60FPS 平滑播放。

音频方面，BGM 使用 `QMediaPlayer` 循环播放，冒险/选关/Boss 战自动切换曲目。脚步和水声为独立 `QMediaPlayer` 实例循环播放。攻击/受伤/POP 声使用短音效播放器。Boss 登场配有独立 `horror` 音效播放器。音量通过 `QAudioOutput::setVolume` 全局控制。

三、小组成员分工情况

本项目由三位小组成员协作完成，具体分工如下：

彭延睿（组长）

- 游戏整体框架设计与搭建：MainWindow 核心控制器、游戏主循环（gameUpdate）、QGraphicsScene/View 场景系统
- 玩家类 Player 全部功能实现：10 种动作状态机、4 种形态能力系统（火/冰/叶/电）、各形态专属技能实现
- 物理引擎与碰撞检测系统：重力/速度物理、Tile 四方向碰撞检测、水中物理/体力系统
- 关卡加载系统 loadLevel()：字符矩阵解析、地图自动优化算法（坑洞填平、长平地缩短）
- 冒险模式地图设计与关卡数据配置
- 游戏存档系统：存档状态的存取（关卡进度、能力列表、HP、体力）
- 项目整体整合、编译调试与版本发布

张良臣

- 三位 Boss 的完整设计与实现：
 - * DukeFishron（猪鲨公爵）：完整三阶段 AI 行为树，螺旋弹幕/直线扇形弹幕算法，地形水转换系统
 - * BrainOfCthulhu（克苏鲁之脑）：飞行追踪 AI，弹幕射击，半血触发二阶段传送突袭机制
 - * IceGod（冰雪之神）：多技能 AI 架构，Xuehua 召唤系统，冰锥扩散/冰束/冰锥雨三种攻击算法
- Boss 登场动画系统（红色遮罩+感叹号）、死亡白屏特效、Boss 血条 HUD
- 弹幕基类 Projectile 的设计与实现（自定义贴图/速度/方向、动画帧、重力、穿透等属性）
- 小怪 MinionEnemy 类的四种属性攻击 AI 行为

王思钺

- UI 系统完整实现：主菜单（含按钮按下状态帧切换）、选关界面（冒险/Boss 子菜单）、暂停菜单、设置界面（音量滑动拖拽调节）、游戏结束界面、剧终动画
- Tile 方块类：8 种方块类型定义、尖刺三角形碰撞体积（shape() 重写）、运动系统（水平/垂直移动刺、伏击刺完整状态机）
- 关卡道具系统：Cake（攻击力加成）、Checkpoint（检查点存档）、Goal（终点通关）、Crate（木箱推动+0.5 秒缓动动画）、Star（Boss 战星星生成）、Badge（Boss 徽章物理掉落）
- 四层视差滚动背景系统（Clouds 1-4）与 Boss 二阶段背景渐变过渡
- 音效多通道系统：BGM 切换逻辑、脚步/水声独立循环音道、短音效触发管理、Boss horror 登场音效
- 综合测试与 Bug 修复

四、项目总结与反思

4.1 项目成果

本项目成功实现了一款功能完整的 2D 横板动作冒险游戏，主要成果包括：

- 完整的游戏循环（60FPS）与 2D 物理引擎
- 可玩的冒险模式（52000 像素长地图）和 Boss 挑战模式（3 位 Boss）
- 6 种角色形态（基础+5 种复制能力）及 8 余种专属技能
- 3 场各具特色的多阶段 Boss 战（总计 8 个阶段的 AI 行为）
- 12 种关卡方块类型及其交互机制
- 7 种敌人类（含 Boss）各自独立的 AI 逻辑和攻击模式
- 完整的 UI 系统（主菜单/选关/暂停/设置/游戏结束/结束动画）
- 音画系统（视差滚动背景、多通道音频管理）
- 采用 Qt6 + C++17，代码量约 5000 行（不含 Qt 框架代码）

4.2 技术反思

(1) 架构设计方面，基于 GameObject 继承体系的面向对象设计有效实现了代码复用，但 MainWindow 作为核心控制器承担了过多职责（状态管理、碰撞检测、UI 控制、音效控制等），其 .cpp 文件超过 4000 行，gameUpdate() 单函数体量巨大。未来可引入组件化架构或实体-组件系统（ECS）来解耦，将不同系统拆分为独立的 Manager 类（如 CollisionManager、BulletManager、AudioManager、SceneManager）。

(2) 关卡地图采用字符矩阵方式定义，虽然简单直观、解析高效，但随着关卡复杂度增加，编辑效率显著降低，缺乏所见即所得的可视化编辑工具。后续可引入 Tiled 等专业地图编辑器导出的 JSON/XML 格式，或开发 Qt 内置的关卡编辑器工具。

(3) 碰撞检测采用逐帧遍历场景中所有物体的方式，在 Boss 战场合弹幕数量较多时可能成为性能瓶颈。今后可引入空间分区优化策略，如四叉树（QuadTree）或网格空间划分（Spatial Hashing）。

(4) 动画系统每帧通过 QPixmap::copy() 从精灵表中切割加载，内存占用较高。可考虑使用纹理图集（Texture Atlas）预加载所有帧并通过 GPU 加速渲染优化，或利用 Qt 的 QGraphicsSprite 等更高层次的动画支持。

(5) 代码中缺乏系统化的日志和调试辅助。在开发后期定位 Bug 时主要依赖断点调试和临时 qDebug 输出。未来可建立统一的日志系统，方便复杂 AI 行为的调试和性能分析。

4.3 协作反思

项目分工按照个人兴趣和技术特点各有侧重：彭延睿侧重游戏引擎核心和玩家系统，张良臣专注 Boss AI 和弹幕算法，王思铖负责 UI/关卡道具和音画系统。三人各有独立的代码模块，降低了直接冲突的概率。

然而实际开发中仍存在较多跨模块交叉修改的情况（如 MainWindow 需要同时整合玩家、Boss、UI 逻辑），后期在代码合并阶段出现了一定量的冲突。今后应在版本管理上加强规范化，采用 Feature Branch 工作流，定期与主分支同步，减少大周期合并时的冲突规模。代码风格和命名规范的统一也需在项目初期确立。

总体而言，三人配合顺利，充分发挥了各自在游戏系统设计、复杂 AI 算法实现和 UI/UX 设计方面的优势，按计划完成了全部功能的开发与集成。通过本次项目，团队加深了对 Qt 框架、C++ 面向对象编程、游戏 AI 设计以及协作开发流程的理解。

4.4 未来展望

本项目尚有以下可扩展方向：

- (1) 增加更多关卡和敌人种类，丰富游戏内容和挑战性
- (2) 引入联网对战或合作通关模式
- (3) 完善剧情系统和对话系统，增加游戏代入感
- (4) 增加成就系统和收集要素（如全部徽章收集奖励）

- (5) 开发内置关卡编辑器，支持玩家自制地图
- (6) 优化移动端触控支持，适配更多硬件平台
- (7) 完善音效资源，增加语音和过场动画

